Studio Nx

Backend(FastAPI/Python) 20 — check_nickname()

2026-10-07

Backend(FastAPI/Python) 20 — `check_nickname()`

항목	내용
파일	`studionx/backend/app/services/auth_service.py`
작업	P1-04 회원가입 — 닉네임 중복 확인 · PRD 01 FR-101 · FR-108
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	19 `check_email()` · 21 `signup()`

1. 하는 일

이 닉네임을 **새로 쓸 수 있는지** 알려 준다. 판단 규칙은 19 와 같다 (`availability_of()` 를 함께 씀).

DB 에 그 닉네임을 가진 행	결과
없음	`{"available": true}`
활동 · 정지 회원	`{"available": false, "reason": "TAKEN"}`
탈퇴 후 3일 안	`{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "…Z"}`
탈퇴 후 3일 지남	`{"available": true}` (가입 때는 18 로 비켜 둠)

대소문자가 달라도 같은 닉네임

이미 있는 닉네임	확인하는 닉네임	결과
`beatmaker`	`beatmaker`	`TAKEN`
`beatmaker`	`BeatMaker` · `BEATMAKER`	`TAKEN`
`beatmaker`	`" beatmaker "` (앞뒤 공백)	`TAKEN`
`café`	`cafe`	`TAKEN` (악센트도 구분 안 함)

2. 언제 쓰나

부르는 곳	`release`	화면
`GET /auth/check-nickname` (24)	`False`	join.html 닉네임 칸의 **Check** 버튼
21 `signup()`	`True`	**Sign Up** 버튼
P1-08 `update_me()`	`False` 또는 `True`	userInfo.html 닉네임 바꾸기 (그때 정함)

3. 코드

import (파일 맨 위)

```
from datetime import UTC, datetime, timedelta

from sqlalchemy import select
from sqlalchemy.orm import Session
```

```
from app.models import User
from app.models.user import UserStatus
from app.schemas.auth import AvailabilityResponse, SignupRequest
```

`auth_service.py` 맨 위의 import 이다 (19 와 같음, 이번에 추가한 것 없음). 이 함수는 **`select`** · **`Session`** · **`User`** · **`AvailabilityResponse`** 를 쓰고, 판단은 19 의 `availability_of()` 에 맡긴다.

함수

```
def check_nickname(db: Session, nickname: str, release: bool = False) -> AvailabilityResponse:
    user = db.scalar(select(User).where(User.nickname == nickname.strip()))
    return availability_of(db, user, release)
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름` 만 가져와 그대로 씀

식	뜻
`nickname.strip()`	앞뒤 공백만 지움. **대소문자는 바꾸지 않음** (이메일과 다름)
`User.nickname == …`	SQL `WHERE nickname = …`. 실제 비교는 **MySQL 이 함** — 아래 정렬 규칙을 따름

MySQL 의 비교 규칙 (정렬 규칙 `utf8mb4_0900_ai_ci`)

`users.nickname` · `users.email` 칸은 이 규칙을 쓴다. 이 DB 에서 직접 확인한 결과다.

비교	결과	뜻
`'Beat' = 'beat'`	같음	`ci` = case-insensitive (대소문자 구분 안 함)
`'café' = 'cafe'`	같음	`ai` = accent-insensitive (악센트 구분 안 함)
`'ａ' = 'a'`	같음	전각 글자도 같게 봄
`'비트' = '비트 '`	**다름**	뒤 공백을 그대로 비교 (`NO PAD`) → 그래서 `strip()` 이 필요

4. 흐름

1. 앞뒤 공백을 지운다.
2. 그 닉네임을 가진 행을 DB 에서 찾는다 (대소문자 · 악센트 무시).
3. 19 의 `availability_of()` 로 판단한다.

5. 왜 이렇게

선택	이유
닉네임은 소문자로 바꾸지 않음	닉네임은 화면에 보이는 이름이다. `BeatMaker` 로 가입한 사람의 표기를 그대로 지킨다
그래도 `BeatMaker` 와 `beatmaker` 는 같은 이름	DB 의 `UNIQUE` 가 이미 그렇게 비교한다. 다르게 확인하면 "✓ Available" 이라고 해 놓고 가입 때 DB 가 거절하는 일이 생긴다. 또 `beatmaker` 를 흉내 낸 `BeatMaker` 계정(사칭)을 막는 효과도 있다

선택	이유
비교를 파이썬이 아니라 DB 에 맡김	확인 결과와 가입 때 DB 판단이 늘 같다
앞뒤 공백은 직접 지움	MySQL 이 뒤 공백을 다르게 본다(`NO PAD`). `"beatmaker "` 로 따로 가입되면 화면에서는 같아 보이는 두 이름이 생긴다
판단은 19 와 같은 함수	탈퇴 3일 규칙 · 정지 회원 처리가 이메일과 똑같아야 한다

6. 시험 (`tests/test_auth_service.py` 에 8개 추가 · 모두 27개, 실제 MySQL · rollback)

시험	확인한 것
없는 닉네임	`available=True`
같은 닉네임	`TAKEN`
공백 · 대소문자 (3개)	`" beatmaker "` · `BeatMaker` · `BEATMAKER` 모두 `TAKEN`
DB 도 같은 규칙인지	`beatmaker` 가 있을 때 `BeatMaker` 로 넣으면 DB 가 거절 (`IntegrityError`) — 확인 결과와 일치
탈퇴 1일	`REJOIN_TOO_SOON`
탈퇴 5일	확인만 하면 그대로, `release=True` 면 `deleted_user_…` 로 비켜 두고 같은 닉네임으로 가입됨

전체 시험 135개 통과, `ruff` 검사 통과. 시험 뒤 남은 행 0개.